

Beyond Link Prediction: Compact Knowledge Representations for Prediction, Retrieval, and Direct Access

Cristian Malaia
cristian.malaia@gmail.com

September 2026

Preprint, draft of 9 September 2026. Not peer reviewed. Preliminary results of research in progress: every number is self-reported from our own runs of the official evaluators, and nothing has been filed on a leaderboard yet, for that reason. Code, logs, receipts and checkpoints: <https://github.com/jinamcg/resonate>; project page: <https://resonate.page>. Two wikikg2 results are excluded from any filing (Section 5, Compliance).

Abstract

How much functionality can be compiled into one knowledge-graph embedding? We take a small representation—a unit-norm complex embedding matrix with block-diagonal relation operators, called ResonatE here and used as the experimental vehicle rather than the claim—and measure what it serves on three Stanford leaderboard tasks and beyond them, on the OGB boards with ten seeds, one test read per run, and pre-registered bars. *Prediction.* On `ogbl-biokg` the bare 27M-parameter embedding scores test MRR 0.8158 ± 0.0006 , above the official 188M-parameter baselines; label-free retrieval members read from the training graph add 0.031 and training-time distillation from ten seeds at temperature 2 adds 0.007, to 0.8528 ± 0.0002 , which would place fifth on the current board at the smallest size on it by $3.5\times$. On `ogbl-wikikg2` a row-sparse training shell fits 2.5M entities in 36 minutes on one GPU; the distilled single model reaches 0.6855 ± 0.0008 , competitive with the strong single models and not compact there (329M parameters). Unseen edges are predicted, not stored: a held-out fact is separated from a random false one at AUROC 0.98 and from the most plausible false alternative at 0.71. *Direct access.* A chain of relation operators compiles to one operator, so a multi-hop question costs one inner-product search at every depth (0.08–0.11 ms, depth 1–6, on a 97k-node graph), an fp16 index over the table is the whole deployable, and every answer is labelled graph-supported or unverified by an exact oracle. *Retrieval from language.* On STaRK-Prime, questions in words over PrimeKG, the same embedding continued on edges and training questions answers a question from its text with the readout that scores a graph query (Hit@1 26.7 ± 0.2 , phrasing gap under one point) while its link MRR rises; with a learned parser head, an exact adjacency walk, a fine-tuned text ranker and a logistic reranker (two pretrained 110M encoders with learned heads around the table, 404M parameters in all), and no generative model at query time, it reaches 43.1 / 68.8 / 75.5 / 54.7 (Hit@1, Hit@5, Recall@20, MRR) on the synthesized test split—first among published rows on Hit@5, Recall@20 and MRR, second on Hit@1 to AF-Retriever [4], which runs a 120B language model on every question—and 28.6 / 53.1 / 61.9 / 40.7 on 98 human-written questions, where the query-time LLM pipelines are far ahead on Hit@1 (58.8) and only Recall@20 is level (the third of four disclosed reads, chosen by a rule fixed before the fourth). A RotatE embedding of the same width and near-equal link MRR takes the identical language projection at 15.8 against 19.7 Hit@1 frozen and 21.2 against 26.9 jointly: equal link prediction does not imply equal usefulness for language. *What did not compile.* Two pre-registered attempts to give the representation an intersection operator failed; real and complex operators tie; block size is flat; a learned combiner that would top the wikikg2 board is reported and not filed because it is fit on validation labels. The observation the evidence supports is that a knowledge representation can be more than a link-prediction model: the same compact representation supports prediction, direct access and, through a

separately trained pipeline of pretrained encoders around it, language-conditioned retrieval without a generative model in the inference path. Code, receipts, checkpoints and every development test read are released.

1 Introduction

A knowledge-graph embedding is usually judged by one number: how well it ranks the missing tail of a held-out triple. This report asks a wider question of one such model. Given a small embedding of entity vectors and a set of relation operators, how much can be served from that representation alone: prediction of edges the graph does not contain, multi-hop access at a cost that does not grow with the chain, and retrieval from questions written in words, without a generative model between the question and the answer?

The vehicle is ResonatE: an entity is a unit-norm vector of $M = k^2$ complex coefficients, a relation and direction is a free block-diagonal operator, unitary at initialisation, a hop applies the operator and renormalises, and the score of a triple is an inner product against the raw target row (Section 2). The operator family is close to existing ones and is not the claim; it is the thing measured. Six pieces of evidence, the OGB ones with ten seeds and one test read per run under a pre-registered protocol (Section 5), the STaRK probe and the control with the seed counts stated where they appear, make the case:

1. *The representation works for graph knowledge.* On Stanford’s `ogbl-biogk` [14] the bare 27.1M-parameter embedding scores test MRR 0.8158 ± 0.0006 , above the official 188M-parameter DistMult, ComplEx and RotatE baselines; with label-free retrieval members read from the training graph and training-time distillation from ten seeds it reaches 0.8528 ± 0.0002 , which would place fifth of twelve on the current board as the smallest entry on it by $3.5\times$. On `ogbl-wikikg2` (2.5M entities) a row-sparse shell trains the embedding in 36 minutes on one GPU and the distilled single model reaches 0.6855 ± 0.0008 , competitive with the strong single models; it is not compact there, and we say so (Section 6).
2. *Missing links are predicted, not stored.* Test MRR is already that measurement; in addition, on 598k held-out Wikidata facts the model’s score separates a true unseen fact from a random false one at AUROC 0.98 and from the most plausible false alternative at 0.71, and 4.18M self-proposed edges at validation-calibrated precision ≥ 0.7 raised every downstream row (Section 8).
3. *Direct access at a cost flat in depth.* A chain of operators compiles to one operator, so a multi-hop question is one inner-product search: 0.08–0.11 ms at every depth 1–6 on a 97k-node graph, against exact traversal that grows with fan-out; an fp16 HNSW index over the embedding (recall@10 0.993–0.994) is the whole deployable; every answer is labelled graph-supported or unverified by an exact oracle (Section 8). The readout is linear in the number of entities or sublinear through the index; what is constant is the cost in depth.
4. *Retrieval from language without a generative model.* On STaRK-Prime [28] the same embedding, continued on edges and training questions, answers a written question with the readout that scores a graph query (26.7 ± 0.2 Hit@1 from the text alone, phrasing gap under one point) and its link MRR rises while it learns; a pipeline of a learned parser head, an exact adjacency walk, a fine-tuned text ranker and a logistic reranker (two pretrained 110M encoders with learned heads around the 37M table, 404M parameters in all), with no generative model at query time, reaches 43.1 Hit@1 on the synthesized test split (second to a 120B-LLM pipeline’s 46.2; first on Hit@5, Recall@20 and MRR) and 28.6 on 98 human-written questions, where those pipelines reach 58.8 (Section 8).
5. *Equal link prediction does not imply equal usefulness for language.* A RotatE embedding of the same width, trained on the same graph under the same regime and tuned to near-equal

link MRR, takes the identical language projection at 15.8 against 19.7 Hit@1 frozen and 21.2 against 26.9 when trained jointly (one seed each, stated as such).

6. *What did not compile.* Two pre-registered attempts to give the representation an intersection operator, one at query time and one by training on conjunctions, did not beat a plain sum of constraint scores; real and complex operators tie; block size is flat above 4×4 ; a learned combiner that would top the wikikg2 board is reported and not filed because it is fit on validation labels (Sections 6.5 and 8).

The path between the boards is part of the evidence. The first `ogbl-biokg` ladder was trained with dense Adam on the table (0.8118/0.8425/0.8505, published 3 September 2026). Scaling that shell to 2.5M entities is where it stops, so it was rebuilt for `ogbl-wikikg2` around a real-view table with row-sparse gradients and row-wise Adagrad [8, 19], $21\times$ faster per step at identical quality, then brought back to the small graph under a pre-registered bar, where it beat the dense ladder on every seed (+0.0040); the distillation temperature found on the large graph ($T = 2$) improved the small one the same way (+0.0019); and the same representation, shell and recipe went to the language benchmark unchanged. The leaderboard positions are validation that the representation is not a toy; the paper’s subject is what it serves.

Relation to prior operator families. Representing a relation as a rotation or a small orthogonal block, so that mixed-relation chains compose non-commutatively, is an established line: RotatE [26] uses 1×1 complex rotations (commuting), QuatE [35] and DensE [22] quaternion rotations, Rotate3D [9] rotations in $\mathbb{R}^3$, OrthogonalE [39] block-diagonal orthogonal matrices with Riemannian optimisation and matrix-valued entities, and Knowledgebra [31] free matrix semigroups. ResonatE’s operator is nearest to OrthogonalE: block-diagonal with small complex blocks, unitary at initialisation—but the constraint is dropped afterwards (the blocks are free), the query side is renormalised after every hop while the target side keeps its norm as a popularity channel, and no path or composition loss is used. We make no claim that this operator is new or better than those families; on `ogbl-biokg` the block size moves validation MRR by +0.0055 ($2 \times 2 \rightarrow 4 \times 4$ at 50k steps) and is flat from 4×4 to 16×16 , which is the size of the effect the operator form has there. The project’s name and the “mode” vocabulary in the code are a coordinate label: the score is invariant under any block-diagonal unitary change of basis ($E \rightarrow UE$, $H_r \rightarrow UH_rU^\dagger$), no Fourier transform is taken anywhere, and nothing in the results depends on the coordinates being spectral. Nor is “complex” a claim: a complex 4×4 block on $\mathbb{C}^{144}$ and a free real 4×4 block on $\mathbb{R}^{288}$ have the same 27,124,129 parameters, and on the 12.5k-step validation screen with two seeds each the real model scores 0.7923 against the complex model’s 0.7897 (a real 8×8 block, which contains the complex 4×4 , 0.7898)—below the 0.005 we had fixed as the smallest meaningful gap, with the sign against the complex form. The complex arithmetic is a way of writing a real block model on $\mathbb{R}^{2M}$, and the released code accepts `real=True` to write it the other way.

2 Model

Entities and relations. Let N be the number of entities and $M = k^2$. The entity table is $E \in \mathbb{C}^{N \times M}$. A query entity is read through $\text{cnorm}(E_h) = E_h / \|E_h\|$; a target entity is read as the raw row E_t , so target norms are free and act as a learned per-entity popularity channel. For each of the $2R$ (relation, direction) pairs (both directions are separate operators) the operator H_r is block-diagonal with M/b independent $b \times b$ complex blocks, initialised as random unitaries by QR decomposition and then trained freely. A hop is

$$z = \text{cnorm}(H_r \text{cnorm}(E_h)), \quad (1)$$

Table 1: The one recipe and its two settings. Everything not listed is shared: unit-norm query, raw target row, free blocks unitary at initialisation, sampled-softmax loss with a $0.1\|z - E_t\|^2$ trajectory term, gradient clipping at 1, cosine schedules, the training shell of Section 2, the members and blends of Section 3, distillation at $T = 2$ (Section 4).

	ogbl-biogk	ogbl-wikikg2
entities / relations / training triples	93,773 / 51 / 4.76M	2,500,604 / 535 / 16.1M
negatives (Evaluator)	500, type-matched	500, uniform
k ($M = k^2$)	12 (144)	8 (64)
block size b	4×4 (36 blocks)	64×64 (1 block)
parameters	27.1M	328.8M
batch / sampled negatives	2,048 / 4,096 (type range)	2,048 / 4,096 (uniform)
head-query fraction	0.5	0.75
table learning rate (RowAdagrad)	0.3	0.6
steps, single model	50,000	800,000
steps, student	50,000	400,000
time per seed on one RTX 5090	4 min (student 8.5)	36 min (student 50)

and the score of candidate t is

$$s(h, r, t) = \text{Re}\langle z, E_t \rangle \cdot e^\tau, \quad (2)$$

with a single learned log-temperature τ . Blocks larger than 1×1 do not commute, so mixed-relation chains are representable, and the operator of a chain is the product of the blocks.

The two dials. k and b are set on validation per graph (Table 1). On `ogbl-biogk`, $k = 12$ and $b = 4$: 27,124,129 parameters counting each complex coefficient as two reals, the convention the board uses for ComplEx (13,562,065 parameter elements). On `ogbl-wikikg2`, $k = 8$ and $b = 64$, i.e. one dense 64×64 operator per relation and direction: 328,842,753 parameters, of which 320M are the table and 8.8M the operators. The reason the dials differ is measured, not chosen (Section 6.3): on the biomedical graph the block-size curve is flat from 4×4 to 16×16 and falls at a dense operator, and width $k = 8$ costs 0.02 whatever the block; on the Wikidata graph the curve rises monotonically to the dense operator, and $k = 8$ with a dense block matches $k = 12$ with 4×4 blocks at 45% of the parameters.

Training shell. The table is stored through its real view as an $(N, 2M)$ tensor and read with a sparse embedding lookup, so a step touches exactly the rows in the batch and autograd emits a row-sparse gradient; reinterpreting the gathered $(B, M, 2)$ rows as complex is free, and every downstream operation is unchanged complex arithmetic. The table is trained with row-wise Adagrad [8, 19]: one accumulator per row, no momentum, untouched rows left bit-identical. The operators and τ use Adam [17] at $5 \cdot 10^{-3}$; a sparse-aware global-norm clip covers both. On `ogbl-wikikg2` at $k = 12$ (a 2.68 GB table) this shell runs at 3.0 ms per step against 64.1 ms for dense Adam on the same batch, with identical validation probes at 1k/2k/3k steps (0.2863/0.3548/0.3706 vs 0.2839/0.3594/0.3719); the dense arm is bound by streaming the table and two moments every step. A self-test asserts the sparse path’s scores and gradients equal the dense parent’s to 10^{-6} . The storage dtype of the table is a separate choice: bf16 storage (rows upcast to fp32 for all arithmetic) costs nothing measurable on `ogbl-wikikg2` at 200k steps (0.6890 vs fp32 0.6879) and halves the table; the rows reported here were trained in fp32 and the released teachers are stored in bf16.

Data pipeline. On `ogbl-biogk` entity ids are per type; we flatten them with per-type offsets so each relation’s head and tail types map to contiguous ranges. A batch is 2,048 triples of one relation (relations sampled by frequency), scored in one random direction against a shared set of

4,096 negatives drawn uniformly from the target type’s range—the Evaluator’s own candidate distribution. On **ogbl-wikikg2** there are no types: batches mix relations, each row has its own direction, negatives are uniform over all N , and the training triples live on the GPU so sampling is three integer draws per step. The head-query fraction (0.75 on **wikikg2**) is a data choice, not architecture: the benchmark’s difficulty is almost entirely head prediction (tail queries 0.936, head queries 0.440 for a single model on validation), and 0.75 is the peak of a curve (0.5: 0.6879, 0.75: 0.6968, 0.9: 0.6893, 1.0: 0.2891 at 200k steps). **ogbl-biokg** is direction-symmetric (0.820 / 0.806) and keeps 0.5.

3 Retrieval members from the training graph

The Evaluator scores each query $(h, r, ?)$ over 501 candidates: the true answer and 500 fixed negatives (and symmetrically for heads). For every candidate c we form members from the *holders* of c under r in the training graph, $\mathcal{H}_r(c) = \{h' : (h', r, c) \in \mathcal{T}_{\text{train}}\}$, and the relation of the query entity h to those holders. All members read training edges only; validation and test edges are never used as graph input.

Both boards.

$$\text{analogy}(h, r, c) = \max_{h' \in \mathcal{H}_r(c)} \cos(\text{cnorm}(E_h), \text{cnorm}(E_{h'}))^3, \quad (3)$$

under the model’s own embeddings, and its top-3 variant (the mean of the three largest similarities). This is case-based reasoning in the model’s own geometry: the embedding compresses an entity’s neighbourhood at training time, the graph remembers it exactly at query time. A candidate with no holders gets the minimum score.

ogbl-biokg. The Jaccard pair replaces the cosine by raw edge-overlap similarity $|\mathcal{N}(h) \cap \mathcal{N}(h')| / |\mathcal{N}(h) \cup \mathcal{N}(h')|$ over typed training-edge sets; it does not use the model and is computed once. Alone on validation the four members reach 0.645 (analogy), 0.751 (top-3), 0.671 (Jaccard), 0.764 (Jaccard top-3). On the dense typed graph these carry the neighbourhood evidence; two-hop members added nothing (held-out $0.8434 \rightarrow 0.8434$).

ogbl-wikikg2. Uniform negatives on a sparse graph mostly fail on “does c take this relation at all”, and the true head of a hub tail (“human”) is a rare entity the table knows little about. Six model-free members are shared by all seeds: *holders* = $\log(1 + |\mathcal{H}_r(c)|)$; over the undirected relation-agnostic training graph (15.2M edges, mean degree 12.2) the common-neighbour count *cn*, its Adamic–Adar weighting *cn_aa* (hubs weigh ≈ 0), *linked* (any other training edge between h and c), Adamic–Adar-weighted length-3 paths *cn3_aa*, and *typed* two-hop paths. Alone they are weak (*cn_aa* 0.096 tail / 0.233 head), together with the per-seed analogy pair they lift a 400k-step model from 0.7001 to 0.7384 held-out. Wikidata hubs with 10^6 holders are capped at 128 holders drawn at random.

Self-augmented members (ogbl-wikikg2). Entities whose citizenship, occupation, native language or birthplace is missing from the training graph give the path members nothing to work with. `augment_graph.py` lets the seed-0 model fill them: for each relation, every plausible head (same “instance of” class as the relation’s training heads) with no edge under it gets the model’s top-1 tail from the relation’s candidate set when its confidence clears a per-relation threshold. The threshold and the set of qualifying relations are calibrated on validation labels (the lowest confidence at which the rule’s cumulative precision on validation rows of that relation is still ≥ 0.7 ; relations with fewer than 50 validation rows are skipped): 18 relations pass and 4.18M edges are proposed, e.g. native language 1.02M at precision 0.70, official language at 0.89. No validation or test edge is added; the members are then recomputed on the training graph plus

these proposals. Base and augmented members are always offered together so the blend can ignore either per relation: with augmented members alone, sport queries fell ($0.511 \rightarrow 0.457$); with both, everything recovers. Effect on the board: row B $0.6820 \rightarrow$ B+ 0.6866 test over ten seeds. This is self-training on the training graph; its calibration is a hyperparameter choice on validation, disclosed as such, and its consequence for the validation numbers is handled in Section 5.

Blend. For each member the 501 candidate scores of a query are z -normalised and combined with one weight vector per (relation, direction) group. On `ogbl-biokg` the vector is *selected* from a fixed list of at most nine candidates (uniform means of the top members and softmax weightings by validation MRR) hierarchically—global, relation family, relation—with a group receiving its own weights only above 4,000 validation rows; a learned combiner gains ≤ 0.0007 there because every member already carries a positive weight. On `ogbl-wikikg2` the weights are *learned*: a listwise logistic regression per group over member z -scores (softmax cross-entropy over the 501 candidates, L2 10^{-3} , global $\rightarrow$ direction $\rightarrow$ relation with warm starts, a group with fewer than 250 fit rows falling back to its direction’s weights). It gains $+0.026$ held-out over selection because `wikikg2`’s members conflict and need negative weights (*holders* is -0.23 on head queries: a candidate that already has many heads is less likely to be the missing one), which selection cannot express. Both procedures are measured cross-fitted—weights fit on a random half of the validation triples, a triple’s two directions kept together, scored on the other half—and then refit on the full split and applied once to test.

4 Distillation at training time

Ten checkpoints are a deployment cost, and a uniform score average of ten `ogbl-biokg` models reaches only 0.8428 (one test shot, retired). Instead a fresh model of the same recipe is trained with the added loss

$$\lambda T^2 \text{KL}(\text{softmax}(\bar{\ell}/T) \parallel \text{softmax}(\ell/T)) \quad (4)$$

on every batch, where ℓ are the student’s logits over [positive | negatives] and $\bar{\ell}$ the mean of the ten frozen teachers’ logits on the same candidates [13], $\lambda = 1$; the teachers are the ten row-A seeds and see only the training triples the student sees. The temperature is the finding. On `ogbl-wikikg2`, $T = 1$ transferred nothing (students 0.6669 and 0.6672 test against a plain seed’s 0.6676) and $T = 2$ transferred $+0.018$ (0.6855 ± 0.0008). Brought back to `ogbl-biokg` under a pre-registered $+0.003$ bar, student-alone validation for seed 0 is $0.8285 / \mathbf{0.8322} / 0.8314 / 0.8300$ at $T = 1/2/3/4$: unimodal with the peak at 2. Our teachers are sharp (the learned scale $e^\tau \approx 9$), so at $T = 1$ the soft targets are nearly one-hot over 4,097 candidates and the ordering among near-miss negatives is invisible; $T = 2$ exposes it, $T \geq 3$ flattens it into the cross-entropy’s way. Ten `ogbl-biokg` students at $T = 2$ reach 0.8321 ± 0.0003 alone on validation, $+0.016$ over their teachers.

5 Protocol

Data and metric. `ogbl-biokg`: 93,773 entities of five types, 51 relations, 4,762,678 training, 162,886 validation and 162,870 test triples; filtered MRR over 500 type-matched negatives per direction. `ogbl-wikikg2`: 2,500,604 entities, 535 relations, 16,109,182 training, 429,456 validation and 598,543 test triples; MRR over 500 uniform negatives per direction. Both via `ogb.linkproppred.Evaluator` (ogb 1.3.6); our loaders assert that the three splits are pairwise disjoint as loaded. MRR ranks the true answer among the sampled alternatives of each query; a value of 0.85 is a rank statistic on that task, not the fraction of arbitrary questions answered correctly.

Pre-registration and test reads. Every configuration’s expected test range was written in the project log before its first test read; every comparison between recipes, members, blend rules, temperatures and shells was made on validation. The test split is read once per run: row A by the trainer’s final evaluation, the blend rows by the committed shot that refits the weights on full validation and applies them once; test score caches are written without computing any per-member test metric. On `ogbl-biokg`, nine development reads were made outside the rows, each a single shot of a configuration chosen on validation and none re-run: the 25k-step baseline (0.8024); 50k steps with 2×2 blocks, seeds 0 and 1 (0.8084, 0.8080); the final recipe’s seeds 0 and 1 trained on a GTX 1080 Ti (0.8134, 0.8123; their RTX 5090 retrains are the first ladder’s row A seeds 0 and 1); the uniform ten-model ensemble (0.8428); seed 0 with three analogy features (0.8368); a 48M single model ($k = 16$, 16,384 negatives, per-entity bias) with its features (0.8472); and the same 48M recipe distilled from twelve checkpoints (0.8535). The sparse ladder added none: its forty runs (rows A, B, C at $T = 1$, C at $T = 2$) are the first and only reads of those runs. On `ogbl-wikikg2`, all seed 0 unless noted: two gap diagnostics (200k steps without N3 0.6637, with N3 0.6487); row B’s first freeze at a blend guard of 4,000 rows on seeds 0, 1 and 5 (0.6785 / 0.6795 / 0.6791), **re-read** after the guard was lowered to 500 on validation (0.6830 / 0.6848 / 0.6805)—the only runs in this report read twice, and row B is not an entry to be filed; $T = 1$ student pilots, seeds 2 and 0 (0.6669 / 0.6672); the ten-seed ensemble with six frozen members (row E, 0.7129) and with augmented and typed members (E+, 0.7164); and a members-only diagnostic (eight shared members under the learned combiner, no model: 0.6723). Each `ogbl-wikikg2` run of the ladder—the ten students’ own final evaluations, the ten C-F blends, the ten leave-one-out ensembles and the full ensemble—was read exactly once.

Compliance. The OGB leaderboard rule reserves the validation set for “standard hyper-parameter tuning (not allowed: gradient-based search, use as model input)” and the test set for final evaluation. The per-relation blend of `ogbl-biokg` rows B and C is a *selection* among a fixed list of at most nine candidate weight vectors per group by validation MRR, which we read as hyper-parameter tuning; those rows are eligible for filing. The `ogbl-wikikg2` learned combiner (`learned_blend.py`) fits about 1,500 per-relation weights by Adam on a cross-entropy loss whose labels are the validation answers, then refits on the full validation split and applies the weights to test: that is gradient-based search on validation labels, and rows F, C-F and the ensemble, which depend on it, are therefore reported with their receipts but not filed. Row B+’s selection blend is also not filed, for the second reason below. What we file for `ogbl-wikikg2` is the distilled single model alone, 0.6855 ± 0.0008 over its ten students’ own test evaluations, whose recipe was tuned on validation only. Second, during `ogbl-wikikg2` development the test split’s relation mix (the query relations, not the answers) was used to explain the validation–test gap, and a validation number reweighted to that mix was used as a development signal afterwards, including for the blend guard change from 4,000 to 500 rows that every later blend row inherits. No test label was used, and the guard sweep itself was run on validation halves, but the decision was informed by test-set statistics; we disclose it here and in the research log rather than claim those rows as clean.

Validation numbers. For rows whose weights are fit on the full validation split we report the in-sample value and the cross-fitted estimate of the same procedure. On `ogbl-biokg` the estimate is within 0.001 of test on every run. On `ogbl-wikikg2` the learned combiner overstates by ≤ 0.001 in-sample (ensemble 0.7998 vs 0.7992 held-out; C-F 0.7880 vs 0.787 over the six seeds whose member caches survived; row F 0.7800 vs 0.779 over seven seeds), and the self-augmentation thresholds are themselves calibrated on validation, so the held-out figures are the ones to be filed. The validation–test gap on `ogbl-wikikg2` (0.70 vs 0.67 for a single model) is a relation shift, not a fit: “languages spoken” is 0.4% of validation queries and 21.5% of test, “native language” 0.1% and 7.4%; reweighting validation to the test relation mix reproduces

Table 2: `ogbl-biokg`, the sparse-shell ladder (to be filed) and the first (dense-Adam) ladder. Ten runs per row, one test read per run. “held-out” is the blend procedure fit on half of validation and measured on the other half; “valid” is the in-sample value. Every row has 27,124,129 parameters.

	test MRR	valid MRR	held-out	hits@1	hits@3	hits@10
<i>sparse shell (to be filed; not yet filed)</i>						
A. single model	0.8158 ± 0.0006	0.8164 ± 0.0006	—	0.746	0.866	0.941
B. A + retrieval features	0.8463 ± 0.0004	0.8472 ± 0.0004	0.8465 ± 0.0004	0.788	0.888	0.952
C. distilled ($T = 2$) + retrieval	0.8528 ± 0.0002	0.8537 ± 0.0002	0.8532 ± 0.0003	0.796	0.893	0.954
C at $T = 1$ (ablation)	0.8509 ± 0.0003					
<i>dense Adam (first ladder, 3 September 2026)</i>						
A. single model	0.8118 ± 0.0013	0.8124 ± 0.0013	—	0.741	0.862	0.939
B. A + retrieval features	0.8425 ± 0.0007	0.8434 ± 0.0007	0.8429 ± 0.0007	0.783	0.884	0.951
C. distilled ($T = 1$) + retrieval	0.8505 ± 0.0004	0.8515 ± 0.0004	0.8512 ± 0.0005	0.794	0.890	0.952

most of the gap (0.6791 vs 0.6678 measured for seed 0).

Statistics. Mean $\pm$ unbiased standard deviation (`torch.std`) over seeds 0–9. Row C’s ten students share the same ten teachers, so their deviation reflects student-seed variance only. The ensemble entry’s statistics are over ten leave-one-out ensembles of nine seeds each, each read once; they share nine of ten members pairwise, so the deviation reflects membership sensitivity only. The full ten-seed ensemble was read once.

Hardware. The sparse `ogbl-biokg` ladder and all `ogbl-wikikg2` rows were trained on rented RTX 5090 (32 GB) instances (torch 2.14, CUDA 13); the first `ogbl-biokg` ladder on RTX 5090 and GTX 1080 Ti (torch 2.6.0, CUDA 12.4); verification on the 1080 Ti. Every rented GPU run of the whole three-day project—both `ogbl-biokg` ladders, every `ogbl-wikikg2` row, students, ensembles, probes and validation sweeps—came to \$52.93 on vast.ai at \$0.60 per RTX 5090 hour (about 88 GPU-hours); the rest ran on a GTX 1080 Ti and a laptop GPU already owned. Checkpoints trained on one machine re-evaluate on the other to within $3 \cdot 10^{-4}$ MRR; the converted sparse-shell seed 0 re-evaluates through the dense code path to its logged test MRR exactly. The biokg members are single-threaded Python: the shared Jaccard pair takes 44 minutes per split once, the analogy pair about 20 minutes per split per seed with a GPU for the cosines and about 6 GB of RAM; the wikikg2 members are GPU-batched and take seconds to minutes per split.

6 Results

6.1 `ogbl-biokg`

Table 2 gives the sparse-shell ladder above the first one. The three clusters of ten do not overlap and each step (+0.031, +0.007) is an order of magnitude larger than the seed noise; the held-out estimate is within 0.0007 of test on every one of the twenty blend runs. The shell is worth +0.0040 on the single model with half the seed spread and +0.0038 with retrieval; with $T = 1$ students the shell’s gain is gone at the top of the ladder (0.8509 vs 0.8505, within noise), and the temperature restores it (+0.0019 on top). Table 3 places the rows: C is fifth of twelve and the fourth single model, 0.0008 behind the fourth entry at one seventh of its parameters; every ResonatE row is the smallest model on the board by $3.5\times$.

Table 3: Public leaderboard for `ogbl-biogk` as fetched on 2026-09-04 [24], and where our rows sit.

	method	test MRR	params
1	RelEns [34] (ensemble)	0.9618	849M
2	ComplEx ²	0.8583	188M
3	UniBi	0.8550	182M
4	AutoBLM-KGBench [38]	0.8536	192M
	ResonatE C: distilled 27M + retrieval	0.8528	27M
5	ComplEx-RP [7]	0.8492	188M
	ResonatE B: 27M + retrieval	0.8463	27M
6	TripleRE [32]	0.8348	470M
7	AutoSF [37]	0.8309	94M
8	PairRE [6]	0.8164	188M
	ResonatE A: 27M single model	0.8158	27M
9	ComplEx [27]	0.8095	188M
10	DistMult [30]	0.8043	188M
11	RotatE [26]	0.7989	188M
12	TransE [5]	0.7452	188M

Table 4: `ogbl-wikikg2`, ten runs per row, one test read per run. “valid” is in-sample for the blend rows; “held-out” is the cross-fitted estimate of the learned combiner (ensemble: the one fit; row F: seven of ten seeds; C-F: the six seeds whose member caches survived). The single-model rows have 328,842,753 parameters.

	test MRR	valid MRR	held-out
A. single model	0.6676 ± 0.0010	0.7030 ± 0.0009	—
B. A + 8 members, frozen blend	0.6820 ± 0.0014	0.7413	—
B+. + self-augmented members	0.6866 ± 0.0015	0.7467	—
F. A + 10 members + learned combiner	0.7222 ± 0.0010	0.7800	0.779
student alone ($T = 2$), to be filed	0.6855 ± 0.0008	0.7190	—
C-F. distilled ($T = 2$) + members + combiner (not filed)	0.7320 ± 0.0010	0.7880	0.787
ensemble of 10 seeds + members + combiner (not filed)	0.7426 ± 0.0002 (LOO)	0.7994	0.7992
full ten-seed ensemble, one read	0.7430	0.7998	

6.2 ogbl-wikikg2

Table 4 gives the ladder. A single 329M model trained in 36 minutes reaches 0.6676; the members and the learned combiner add +0.055; the $T = 2$ student adds +0.010 more; the ten-seed ensemble under the same combiner +0.011 beyond that. Table 5 places the rows against the public board’s top entries. The entry to be filed, the distilled single model alone at 0.6855, would sit between TripleRE + NodePiece (0.6866) and InterHT (0.6779), eleventh of twenty-eight. The two combiner rows are shown for what they measure and are not filed (Section 5): the ensemble would be first and is also the largest entry on the board (ten tables, 3.29B parameters); the single model C-F would be second.

6.3 Why the dials differ

Table 6 is the evidence. On `ogbl-biogk`, width is the binding lever ($k = 8$ costs 0.02 at every block size) and the block size is flat from 4×4 to 16×16 at both widths, then falls at the fully dense operator (-0.013 at $k = 8$). On `ogbl-wikikg2`, capacity in the table is expensive (each unit of k costs millions of rows) and the operator is cheap (535 relations), so larger blocks keep paying all the way to dense, and $k = 8$ with a dense block matches $k = 12$ with 4×4 blocks at 45% of the parameters. One shared setting would cost one of the graphs; the shared setting closest to free is $k = 12$ with 16×16 blocks (biogk 0.7891 vs 0.7893; wikikg2’s best $k = 12$ arm),

Table 5: Top of the public leaderboard for `ogbl-wikikg2` as fetched on 2026-09-04 [1] (28 entries), and where our rows sit.

	method	test MRR	params
	ResonatE ensemble $\times 10$ + retrieval (not filed)	0.7426	3.29B
1	RelEns [34] (ensemble)	0.7392	2.18B
	ResonatE C-F: distilled 329M + retrieval (not filed)	0.7320	329M
2	StarGraph + TripleRE + Text [20]	0.7305	1.93B
3	InterHT+ [33]	0.7293	156M
4	StarGraph + TripleRE [20]	0.7286	93M
5	InterHT+ (256-dim)	0.7257	148M
6	StarGraph + TripleRE (2022)	0.7201	87M
7	CompoundE3D [10]	0.7006	751M
8	TranS [36]	0.6939	38M
9	TranS (2022)	0.6882	19M
10	TripleRE + NodePiece	0.6866	36M
	ResonatE: distilled 329M, alone (to be filed)	0.6855	329M
11	InterHT	0.6779	19M
14	ComplEx-RP (50-dim) [7]	0.6392	250M
23	ComplEx (250-dim)	0.4027	1.25B

Table 6: Block-size and width curves on validation, seed 0. Left: `ogbl-biokg` at the 12,500-step screen (dense-Adam shell). Right: `ogbl-wikikg2` at 200k steps.

ogbl-biokg			ogbl-wikikg2		
k , block	params	valid	k , block	params	valid
8, 2×2	12.0M	0.7616	8, 8×8	321M	0.6876
8, 4×4	12.1M	0.7677	8, 16×16	322M	0.6892
8, 16×16	12.2M	0.7677	8, 64×64 (dense)	329M	0.6945
8, 64×64 (dense)	12.8M	0.7548	12, 2×2	721M	0.6818
12, 4×4	27.1M	0.7893	12, 4×4	721M	0.6879
12, 8×8	27.2M	0.7904	12, 8×8	723M	0.6937
12, 16×16	27.5M	0.7891	12, 16×16	725M	0.6961
16, 4×4	48.2M	0.7928	12, 144×144 (dense)	765M	0.6907
			16, 2×2	1.28B	0.6966

which we did not run as a submission.

6.4 How the recipe was found

ogbl-biokg, first ladder. The starting point was a hand-picked recipe from smaller graphs: $k = 12$, 2×2 blocks, 4,096 negatives, learning rate $5 \cdot 10^{-3}$ cosine, 25,000 steps. Each default was treated as a hypothesis under a tiered protocol written down before each run: a 12,500-step *screen* on validation, promotion only above the current recipe by +0.004 MRR, then a 50,000-step *confirmation*. Sixteen levers went through it; two survived. *Budget*: validation MRR at 6,250 / 12,500 / 25,000 / 50,000 steps is 0.7351 / 0.7829 / 0.8032 / 0.8085, and the gain per doubling shrinks fast enough that 100k projects at $\leq +0.002$, so the budget was doubled once and frozen. *Capacity*: in 2,000-step probes validation MRR grows log-linearly in k with no plateau, but the gain compresses with budget— $k = 16$ ’s +0.025 at 2k steps is +0.003 at 50k, and even $k = 32$ (192M) misses the bar—whereas $2 \times 2 \rightarrow 4 \times 4$ blocks (118k parameters) keep +0.0055 at 50k. The remaining fourteen levers, mostly the tricks other entries rely on, are in Table 7 (Section 6.5).

ogbl-wikikg2. The biokg recipe was run unchanged on the large graph through the new shell (one day, one RTX 5090): a fail-fast ladder of 3k / 20k / 100k / 200k / 800k steps reached TripleRE’s level after 67 seconds of training (0.5792 at 20k) and 0.7041 validation at 800k. The dials were swept at 200k (Table 6); the head-query fraction was swept (Section 2); relation-balanced sampling was rejected (up-sampling a relation with 56k triples memorises it: 0.6871 vs 0.6945); N3 lowered both splits equally; the blend guard was lowered from 4,000 to 500 rows after a monotone held-out sweep (0.7392 $\rightarrow$ 0.7411, the test-dominant relations had been denied their own weights); the members were added in the order holders/analogy (0.7001 $\rightarrow$ 0.7224 held-out), two-hop (0.7349), three-hop and a direction level (0.7384), self-augmented (0.7502 with base and augmented offered together), typed paths; the learned combiner replaced selection (0.7499 $\rightarrow$ 0.7791 held-out for the single model); $T = 1$ distillation failed and $T = 2$ succeeded. A compositional entity table (rows built from neighbours through the inverse operators) was piloted on both graphs and rejected: neutral with the table kept, -0.1 to -0.17 without it.

Back-transfer. Two things came back to **ogbl-biokg** under pre-registered bars. The shell: with the row-A recipe unchanged, validation went from 0.8124 ± 0.0013 (dense Adam, ten seeds) to 0.8158 ± 0.0005 (three seeds at table learning rate 0.3; 0.8135 at 0.6), so a ten-seed test row was run with the bar “mean ≥ 0.8143 ”—met by the mean and by every seed. The temperature: a single $T = 2$ student beat the $T = 1$ student by $+0.0037$ on validation against a $+0.003$ bar, and $T = 3$ and $T = 4$ were below $T = 2$. Two things did not: the learned combiner ($\leq +0.0007$ on biokg, whose members never conflict) and two-hop members (no change on a dense typed graph).

6.5 What did not compile

Two patterns account for the rejected **ogbl-biokg** levers (Table 7; the 12,500-step reference is the 4×4 recipe’s own 0.7893). First, *the architecture pre-empts the trick*: a relation-prediction auxiliary loss [7] is monotonically harmful because type-grouped batches already force the model to separate relations; N3 regularisation [18] has a real but tiny interior optimum because renormalising after every hop already disciplines magnitudes; compositional path training [11] has nothing to repair because hops compose exactly by construction and the chains only spend the update budget. Second, *screen-tier wins vanish at the plateau*: more negatives and tied reverse operators both pass the screen and are flat or negative at 50k—they speed convergence without raising the ceiling. A one-cycle schedule loses to cosine, and phase-only (unitary) relation operators never reach 90% accuracy at any depth in the synthetic chain testbed: the moduli off the unit circle are load-bearing.

Where the errors are. On **ogbl-biokg**’s rarest relations the lookup beats the embedding outright: on side effects with at most 19 training edges the analogy feature alone reaches held-out MRR 0.0175 against the 27M model’s 0.0147 (blend 0.0196). Of the remaining errors 36% are drug $\rightarrow$ side-effect tails, where even a 22-member ensemble scores 0.17—a floor set by the data. Across the ten seeds the truth-versus-runner-up margin has a constant spread of 0.28 and only its mean moves: seeds disagree on which near-tied candidate wins, not on what they know, which is why more seeds were never going to close the gap and why the deployable unit is one table plus a lookup. On **ogbl-wikikg2** 92% of the validation deficit is on head queries, and the members that help most are the ones that answer “who speaks language X” through person $\rightarrow$ citizenship $\rightarrow$ country $\rightarrow$ official language, which is exactly the two-hop path member.

Table 7: Levers rejected on `ogbl-biokg` validation (dense-Adam shell). Screen: 12,500 steps on the 4×4 recipe (reference 0.7893); confirm: 50,000 steps (reference 0.8140). “✓” marks a passed screen.

lever	screen	confirm	verdict
relation-prediction aux loss, $\lambda = 0.25/1/4$	0.7886/0.7849/0.7741	—	monotone negative; type-grouped batches
N3 regularisation, $w = 10^{-3}/10^{-2}/10^{-1}$	0.7902/0.7912/0.7894	—	already separate relations
compositional (path) training, $p = 0.25/0.5$	0.7711/0.7475	—	real interior peak, half the promotion bar
16,384 negatives	0.7963 ✓	0.8148 (+0.0008)	hops compose exactly
tied reverse operators (adjoint)	0.7936 ✓	0.8128 (−0.0012)	already; chains tax the update budget
one-cycle schedule, max lr $3 \cdot 10^{-2}$	0.7474 [†]	—	convergence-speed effect
uniform 8×8 blocks	0.7904	—	reverse blocks are not slack
unitary (phase-only) relations	< 90% at every depth (synthetic)	—	update count is the bottleneck
real-valued model, equal parameters	0.7923 (2 seeds)	—	flat; 4×4 is the knee
50k budget (adopted)	—	0.8085 vs 0.8032 (25k)	off-circle moduli are load-bearing
4×4 blocks (adopted)	0.7893 ✓	0.8140 ✓	“complex” is not a claim
sparse shell (adopted)	—	0.8158 vs 0.8124 (3 vs 10 seeds)	

[†]screened on the 2×2 baseline against its own cosine 0.7829.

7 Compressing the entity table after training

Ninety-seven per cent of the biokg model is the entity table: $93,773 \times 144$ complex rows against 117,505 parameters of operators and temperature. If a trained table carries redundancy, it can be removed *after* training, without touching the recipe that produced it. This section reports that it can, at a cost that does not vary from seed to seed.

Construction. Entities are grouped into tiers by training degree. Inside tier t the rows are read through their real view, k -means clustered into K_t groups, and each cluster is replaced by its top $d_t = 2w_t$ principal directions: an entity stores a narrow coefficient vector and the tier stores K_t shared projections P and offsets μ , so

$$\text{row}(e) = \mu_{c(e)} + \text{coef}(e) P_{c(e)}, \quad (5)$$

with $c(e)$ the fixed cluster of e . The table costs $\sum_t [n_t d_t + K_t(d_t + 1)2M]$ real parameters. Operators and temperature are copied from the trained model and frozen. **Nothing is trained:** the construction is k -means plus a per-cluster eigendecomposition and takes about ten seconds on a GTX 1080 Ti. At full width with $K = 1$ it reproduces the source model exactly, which is the sanity check.

The graph decides whether this works. On `ogbl-wikikg2` the same construction with one subspace per tier collapses (0.4834 against the student’s 0.7190): the trained tail rows of a 2.5M-entity table are close to isotropic, and there is no shared narrow subspace to project onto. On `ogbl-biokg` the same one-subspace-per-tier point reads 0.7180 against 0.8155 — a real loss, but not a collapse — and clustering recovers most of it. Two properties of the graph explain the difference. biokg has $27\times$ fewer entities, so a bank of K subspaces is $27\times$ more expensive per

Table 8: **ogbl-biokg**, post-hoc tiered compression of the row-C distilled students. Same seeds, same members, same blend procedure; one test read per seed. “model alone” is the held-out blend of the model’s own scores without members.

	params	model alone	test MRR
C. distilled ($T = 2$) + retrieval	27,124,129	0.8305	0.8528 ± 0.0002
C’. compact table + retrieval	9,555,497	0.8162	0.8468 ± 0.0003
cost of $2.84\times$ compression	-64.8%	$+0.0143$	$+0.0061 \pm 0.0004$

row: at widths (2, 4, 8, 32, 96, 144) the wikikg2-optimal $K = 4096$ would cost 56.7M parameters, more than twice the 27.0M table it compresses, and the knee instead sits near $K = 256$. And biokg’s entities live in five type-disjoint ranges with type-matched negatives, so the model never has to separate a protein from a side-effect; its rows are correspondingly more clusterable.

Where the width belongs. Degree tiers on biokg are far flatter than on wikikg2 (median degree 22 against ≈ 6), and the answers concentrate at the top: the 24% of entities with degree < 5 are 0.5% of validation answers, while the 1.4% with degree ≥ 1024 are 44%. Width spent on the tail is close to wasted. Two configurations make the point: (4, 8, 16, 36, 72, 144) at 11.0M scores 0.8001 while (2, 4, 8, 32, 96, 144) at 8.5M scores 0.8006 — 2.5M fewer parameters for the same accuracy, because the first widens three tail tiers holding 6.6% of the answers and starves the tier holding 30%. Holding the top two tiers at full width and cutting the tail to (2, 4, 8) gives the configuration used below: widths (2, 4, 8, 48, 144, 144), $K = (256, 256, 256, 32, 1, 1)$, 9,555,497 parameters, 35% of the original.

Result. Table 8 applies this to the row-C distilled students. The compact row is not a separately trained model: it is row C’s released checkpoint with its table replaced, blended with the same four label-free members.

Two things are worth stating precisely. First, **the members absorb most of the loss**: the narrow table gives up 0.0143 on its own scores and 0.0061 after blending, so 57% of what the wider table contributed is recovered by features that read the training graph rather than the table. The same ratio appears when the construction is applied to the row-A/B model instead ($0.0093 \rightarrow 0.0028$), from a different source model. Second, **the cost does not vary across seeds**: the per-seed gap spans 0.0056–0.0067 over ten seeds with a single mild outlier (seed 5), and the compact row’s seed spread (± 0.0003) is within a whisker of the uncompressed row’s (± 0.0002), so the compression adds no variance of its own. No seed produced a compression that failed. This is the practical claim: the reduction is a property of the trained table, not a lucky draw, and it can be applied to a released checkpoint without rerunning anything.

Row C’ is proposed alongside row C rather than in place of it. The two are one model at two operating points, not two results: C’ is C’s released checkpoint with its table replaced, so it inherits C’s recipe and C’s errors, and the pair must not be read as independent evidence. On the board of Table 3 C’ would sit sixth, 0.0024 below ComplEx-RP and 0.0120 above TripleRE — a model $49\times$ its size — at $9.8\times$ fewer parameters than AutoSF, the smallest entry currently listed, and $20\times$ fewer than the leading single models.

A refit is free in parameters, and closes the gap (one seed, validation only). The compact table above is an initialisation: k -means plus a per-cluster PCA, never trained. Training the coefficients and the subspace banks — with the operators, the temperature and the cluster assignment frozen, so the parameter count cannot move — recovers most of what the compression cost. On seed 0, 50,000 steps at learning rate 5×10^{-4} take the model’s own validation MRR from 0.8176 to 0.8301 against the wide model’s 0.8322, i.e. 86% of the model-alone deficit, and the blended held-out estimate from 0.8478 to **0.8539** against row C’s 0.8532 — parity at 35%

of the parameters. The rate matters more than the schedule: at the ladder’s own 5×10^{-3} the run collapses to 0.7196 by 10k steps and spends the rest of the cosine climbing back to 0.8189, a gain of +0.0013.

We report this as a validation-only result on one seed and do not file it. The +0.0007 over row C is well inside the ± 0.0002 – 0.0004 seed spreads and must not be read as the smaller model winning; the claim is parity. No test read has been taken for the refit, and the filed row C’ remains the init-only model at 0.8468 ± 0.0003 . It also corrects the reading we drew from the absorption ratio: 57% absorption bounds what a model improvement is *worth* after blending, not how large that improvement can be, and a refit worth +0.0125 on the model survives it.

Disclosure. The construction is deterministic on a given device — re-running it reproduces the coefficients, bases and cluster assignments bitwise — but it is *not* portable across devices: k -means seeds its centroids from the device RNG, so a CPU run and a GPU run of the same checkpoint produce different clusterings and tables differing by 49% relative. The two are equally good (validation 0.8197 against 0.8176), so the method reproduces even though the artefact does not; but the compact checkpoints must therefore be released as files, and a re-derivation from the script alone will land near, not on, the committed numbers. The k -means seed is fixed at 0 throughout and is not selected on validation; the clustering-initialisation spread is about 0.002, a third of the reported gap, so selecting it would inflate the result by a meaningful fraction of what is being claimed.

8 Beyond the boards: missing links, direct access, language

The boards are one-hop scores. This section measures what else the same representation serves, in the order of the introduction: prediction of edges the graph does not contain, access at a cost flat in depth, and retrieval from questions in words.

Depth. On synthetic worlds with exact ground truth at every depth, models trained only on chains of depth ≤ 4 ($k = 8$, 25k steps, three seeds, 2,000 chains per depth) are read out to depth 48. Three results, in decreasing order of what they show. *One relation applied n times*, on 1,000 entities with seven random permutations as relations: the diagonal model holds 0.987 at depth 8 and crosses 90% between depth 10 and 12 and 50% between 16 and 20; the 4×4 -block model 0.963 at depth 8. This is a statement about powers of a single operator, not about composition. *A different relation at every hop*, so that order matters, on the same random permutations: neither the block model nor a dense 64×64 operator learns depth 2 (0.21 and 0.65 accuracy, ≈ 0 beyond), because random permutations share no low-dimensional structure; the block model does not compose arbitrary relations. *A different relation at every hop on relations that form a group*: a turtle on a 16×16 torus with forward, back, turn left and right, strafe left and right, and U-turn, i.e. 1,024 (heading, x , y) states and rigid motions generating $\mathbb{Z}_{16}^2 \rtimes \mathbb{Z}_4$, whose irreducible representations are at most four-dimensional. Half of all depth-2 chains change their answer when reordered, and a commuting diagonal model reaches 0.75 at depth 2. The 4×4 -block model trained on depth ≤ 4 reaches 0.949 at depth 8, crosses 90% between depth 10 and 12 and 50% between 20 and 24 (seed s.d. ≤ 0.003 to depth 8); the dense operator reaches further (0.995 at depth 8, 0.885 at depth 20), so the block size is a ceiling rather than the mechanism. The allowed statement is therefore that the model composes non-commuting relations beyond its trained depth when the relations form a group with a low-dimensional representation, and not for arbitrary permutations; whether the relations of a real graph are closer to the former or the latter is not measured. Both mixed-relation experiments were pre-registered after a first external review asked for them. Mapping natural-language questions onto the table is the subject of the last paragraph of this section.

Latency, flat in depth, and a compiled index. Because the per-hop renormalisation is a scalar, a chain of relation operators collapses into one block-diagonal operator (2,304 bytes for the 2×2 models; a few small block products per question template, computed once) and the readout is a real inner product—maximum inner-product search over a real $N \times 2M$ table. The constant cost below is therefore the cost *after* the chain is compiled, and it is the model’s readout alone: the oracle lookup that labels an answer is a separate graph query, timed only in the Hetionet tool (0.4–0.8 ms per query on its 47k-node graph) and not on the larger graphs, and the retrieval members of Section 3 need the holder lists of a query’s candidates at query time; the complete verified-answer workflow has not been measured end to end. On DRKG [15] (97,238 nodes, 5.87M edges, a 28.1M-parameter model) batched multi-hop queries take 0.08–0.11 ms at every depth 1–6 while exact graph traversal grows with fan-out, up to $45\times$ slower at depth 5–6; compiled scores equal hop-by-hop scores to 10^{-5} with identical top-10 sets over 1,867 queries. An HNSW index [16, 23] over an fp16 copy of the table gives recall@10 0.993 against the exact readout on DRKG (848 bytes per row including links) and, on ogbl-wikikg2’s 2,500,604 rows (1.98 GB, 852 bytes per row, built in 673 s on 64 cores), 0.994 at efSearch 512 with 2.2 ms per single query on CPU; the exact GPU readout over all 2.5M rows is 2.4–3.3 ms. Storing the table in fp16 leaves the top-10 identical on all 3,334 queries over two biomedical graphs; int8 costs a little (top-1 agreement 0.98), int4 breaks retrieval (0.70). The hub-heavy norm distribution of ogbl-wikikg2 needs the standard inner-product-to-cosine augmentation (an extra coordinate $\sqrt{R^2 - \|x\|^2}$); a raw inner-product build lost hub rows (recall@10 0.864).

Oracle validation and new edges. On Hetionet [12] and DRKG every top answer is checked against an exact graph oracle and labelled *graph-supported* (a supporting path exists in the training graph, which can itself be incomplete, outdated or wrong) or *unverified prediction* (unsupported by that graph; not a discovery)—[ok] and [NOVEL] in the tool’s output; on the trained metapath libraries the top answer is oracle-verified 94.6–100% of the time across all ten DRKG paths and 99–100% on all eight Hetionet paths. On ogbl-wikikg2 the same readout with the same flags: for Douglas Adams, “notable work” (two training edges) returns *Last Chance to See*, *The Restaurant at the End of the Universe*, *The Salmon of Doubt*, *Mostly Harmless* and *The Hitchhiker’s Guide* in the top ten, nine of them absent from every split; “educated at” returns Cambridge and St John’s (training edges) and then Harvard, Trinity, York and Oxford as novel. The self-augmentation of Section 3 is the same mechanism run at scale with a validation-calibrated threshold: 4.18M proposed edges whose per-relation precision on held-out validation rows is ≥ 0.7 , and which raised every downstream row. Fine-tuning 496 held-out *treats* edges into a trained DRKG model took 1,500 steps, 223 s of wall-clock (3.7 minutes) on a single laptop GPU, absorbed all of them to rank 1 (MRR 0.147 $\rightarrow$ 1.000) with a worst metapath regression of -0.008 .

A claim checker, measured. The reviewer’s question of what the labels are worth was answered with a pre-registered measurement on the ogbl-wikikg2 test split, whose 598,543 facts are all absent from the training graph, so the oracle calls every one of them unverified and only the model’s score can separate them from false claims. Against a corrupted claim with a random entity, the distilled single model’s score separates true from false with AUROC 0.982 (0.997 for tail-side claims, 0.952 for head-side); calling a claim supported when it ranks first among the 501 candidates accepts 62% of true unseen facts and 0.1% of random false ones (rank ≤ 10 : 82% and 1.8%). Against the most plausible false alternative, the highest-scoring of the 500, the separation is weak (AUROC 0.709; 0.933 tail-side, 0.347 head-side): the checker rejects random errors reliably and cannot adjudicate near-miss candidates, and the head direction, where the answer is a rare entity, is its weak side. The claims must already be graph triples; text has not been run through it. (wikikg2/claim_check.py; the row-A teacher gives 0.977 / 0.686.)

Training cost and memory. A training step touches the rows in its batch whatever N is, and the optimiser keeps one float per row. **ogbl-biokg**: 50,000 steps in 231 s on one RTX 5090 (18 minutes on a GTX 1080 Ti with the dense shell); a student with ten teachers in 506 s; the table is 108 MB in fp32 and 54 MB as the fp16 serving table (fp16 serving was validated on the biomedical graphs above; bf16 training was not run on this graph). **ogbl-wikikg2**: the 329M model trains 800,000 steps at 2.7 ms per step, 36.5 minutes, from a 1.19 GB fp32 table (0.6 GB in bf16, measured at no cost); a student with ten bf16 teachers resident, 400,000 steps at 7.6 ms, 50 minutes; the $k = 12$ table (2.68 GB) trained at 3.0 ms per step in 15.7 GB of GPU memory, against 64.1 ms for dense Adam. Row-wise Adagrad costs nothing measurable on the biomedical graphs either (Hetionet hits@10 0.782 vs 0.782 over three paired seeds; DRKG within one standard error). From the measured per-row costs, 100M entities at $k = 12$ would need a 115 GB fp32 training table, 0.4 GB of optimiser state, an 85 GB fp16 HNSW serving index (or about 5 GB of RAM plus NVMe in a DiskANN layout [25]) and roughly 7 GPU-hours for 20 epochs of 500M edges on one 5090-class card; that remains a projection, and none of it has been run.

Language into the table (STaRK-Prime). STaRK [28] asks written questions over PrimeKG (129,375 nodes, 18 relations, 8.1M edges) and scores the ranked entities: Hit@1, Hit@5, Recall@20 and MRR over the whole node set, on a synthesized test split (2,801 questions, and a 280-question subset) and 98 human-written questions. This probe lives in a separate repository (<https://github.com/jinxmccg/resonate-stark>) with its own pre-registrations and reads; it is reported here for what it says about the table. The same architecture ($k = 12$, 4×4 blocks, the sparse shell) was trained on the graph’s edges (held-out link MRR 0.557) and then continued on edges and 12,324 training questions together, a question entering as a vector from a small head on a fine-tuned 110M text encoder and scored by the same readout $\text{Re}\langle z, E_t \rangle$ that scores a graph query. The table got better at links (0.568, three seeds, s.d. 0.001) and answers a question from its text alone at Hit@1 26.7 ± 0.2 on synthesized wording and 25.7 ± 0.4 on human-style paraphrases (validation; the paraphrases are written by a local 7B model from the question text only and stand in for the human set, which was never opened during development). A learned three-headed parser (answer type; anchor entities by nearest neighbour in the table; relation operators), trained in four minutes on weak labels derived by walking the graph, reads questions into the model’s query space at 25.6 ± 0.6 / 22.3 ± 0.4 Hit@1 on the graph path, against 24.9 / 20.9 for a hand-written parser backed by the 7B model. With the same parser and anchors, the exact adjacency walk alone scores 18.9, the model’s composed operators alone 19.0 and both 24.9: the walk says which candidates are graph-supported and the model says in what order. The reference system (P3 in the probe’s record) runs no generative language model at query time; what does run is two 110M transformer encoders (the fine-tuned text ranker, and the encoder under the parser head and the table readout), the adjacency walk, the model and a logistic reranker fit on train with out-of-fold features, at 32 ms median (49 ms p90) per question at batch size one on one RTX 5090 in 1.9 GB of GPU memory (measured in the probe’s repository). With the 7B parser instead it is two Hit@1 points higher on validation, and both are reported. The published rows it is compared with are the leaderboard’s own [28] and the later Prime results the leaderboard does not carry: KAR [29], mFAR [21], 4StepFocus [2], FocusedRetriever [3] and its published successor AF-Retriever [4], each copied from its own table. Their regimes differ (mFAR is trained on the STaRK train split and runs no LLM at query time; KAR, 4StepFocus, FocusedRetriever and AF-Retriever are zero-shot pipelines that run GPT-4o, Llama 3.3 70B or gpt-oss-120b on every question), and two of them, 4StepFocus and FocusedRetriever, evaluate only on the 280-question 10% subset of the synthesized split, so they are compared with the 10% read and never with the full one; on 280 questions a 95% interval on Hit@1 is about ± 6 points. The human set has 98 questions, so its bootstrap intervals are wide (reported in the repository) and no superiority claim is made on it beyond what those intervals allow. Committed reads,

each one frozen pipeline run once per split after a dry run that reproduced the validation number to four decimals. The reference read (P3): synthesized 43.1 / 68.8 / 75.5 / 54.7, the 10% subset 41.8 / 72.5 / 77.8 / 54.3, human-written 28.6 / 53.1 / 61.9 / 40.7 (Hit@1 / Hit@5 / Recall@20 / MRR). The best published number in each column is 46.2 / 63.7 / 68.3 / 54.0 on the full split (AF-Retriever; Recall@20 from mFAR, which reports 40.9 / 62.8 / 68.3 / 51.2 with no LLM at query time), 46.4 / 63.9 / 65.5 / 53.7 on the 10% subset (FocusedRetriever) and 58.8 / 69.4 / 59.9 / 62.7 on the human set (FocusedRetriever, AF-Retriever, KAR, AF-Retriever). Read plainly: on the synthesized splits the pipeline is first on Hit@5, Recall@20 and MRR and second on Hit@1 to a pipeline with a 70B–120B language model per question; on the human set those pipelines are far ahead on Hit@1 and only Recall@20 is level—the parser is trained on synthesized templates and does not read free phrasing the way a language model does. The test splits were read four times in all and every read is disclosed (Hit@1 on the synthesized / 10% / human splits): P1, a hand-written parser and an off-the-shelf embedder, 28.7 / 28.2 / 20.4; P2, the pipeline above, 41.8 / 41.8 / 30.6; P3, P2 plus one correctness fix (gene symbols shorter than four characters were never matched by the parser), 43.1 / 41.8 / 28.6; P4, P3 plus a learned selector over four readings of each question, 43.7 / 41.8 / 25.5. P3 is the reference by a rule fixed before P4 was read: the system put forward is the best model and architecture, not the best score, so a correctness fix stays and a selector fit to the synthesized templates, which cost five of the 98 human questions, goes. On 98 questions the differences among P2, P3 and P4 are two to five questions, inside the noise. A fifth candidate, letting the question vector disambiguate same-name entities before the walk (latent-confirmed anchors), lost 1–7 Hit@1 at every threshold on training questions and was dropped without a read. What this does and does not show: the margin over the leaderboard’s own rows on synthesized questions comes largely from ordinary pipeline work (a fine-tuned embedder, a reranker); what is the model’s own is that one table, one readout, serves link prediction, the oracle and language questions, with a sensitivity to phrasing under one point, and that a 26-point standalone answerer from a question is its recall ceiling (≈ 36 Recall@20), since a single readout has no exact AND over constraints—the walk supplies that. Bolting an AND onto the readout at query time does not help: over the parsed constraints, min, soft-min, a product of per-constraint probabilities and a soft count of satisfied constraints all score below the plain sum on validation (13.3–16.5 against 19.0 Hit@1, model alone), because the single-hop table was never trained on conjunctions and the parsed constraint set is noisy. Training the table on 30,000 sampled conjunctions (pre-registered) doubled its scores on held-out conjunctions, but on real questions the plain sum still won; the exact walk remains the intersection operator (Section 6.5).

9 Limitations and disclosures

The blend weights of the retrieval rows are selected or fit on the full validation split; we report the in-sample number and the cross-fitted estimate, which is within 0.001 of test on `ogbl-biokg` and within 0.001 of in-sample on `ogbl-wikikg2`, where the test split’s relation mix differs from validation’s and no validation number predicts test. The self-augmentation thresholds on `ogbl-wikikg2` are calibrated on validation labels; no validation or test edge is used as input, but the augmented rows’ in-sample validation numbers inherit that calibration. Three of the ten `ogbl-wikikg2` students (seeds 0, 2, 3) were deleted by the campaign script after their reads and cannot be re-verified from a checkpoint; their receipts and logs are released. Row C’s students share one teacher pool on both boards; the leave-one-out ensembles share nine members pairwise. The `wikikg2` ensemble entry is ten 329M tables, the largest entry on that board. At inference a retrieval row needs the holder lists of a query’s candidates—cheap, but a graph lookup the pure embedding does not need. Every development test read is listed in Section 5, including the three runs of a row not to be filed that were read twice. Training is seeded but CUDA kernels are not bit-deterministic, so a retrained checkpoint reproduces its MRR to seed-level noise rather than

bit-for-bit; the blend stage is exact. No external data was used on either board. On STaRK-Prime the test splits were read four times in total (all four reported above), the human-written set never served development, a 7B instruction model was used offline for paraphrases and weak labels only. A reviewer’s control was run: RotatE (its L_2 variant) trained on the same graph at the same width (288 numbers per entity) under the same regime and tuned by held-out link MRR only (0.549 against 0.557), with the identical projector on top. Frozen, its table takes the projection at Hit@1 15.8 against 19.7 (inner-product readout on both; 15.5 with RotatE’s own distance readout); trained jointly on edges and questions, 21.2 against 26.9, and joint training raised RotatE’s link MRR by 0.003 against 0.013 here. One seed each, one dataset; a first pass with an untuned RotatE showed a two-fold gap and is retracted in the probe’s record. The allowed statement is that this table takes a language projection better than a RotatE table under the same recipe, not that only it can.

10 Conclusion

One small embedding of entity vectors with block-diagonal relation operators, trained on edges, serves more than the link-prediction score it is usually judged by. It predicts edges the graph does not contain, separating true unseen facts from random false ones reliably and from near-misses weakly; it answers multi-hop questions at a cost flat in depth, from an index that is the whole deployable; and, continued on training questions, it answers questions written in words with the same readout, inside a separately trained pipeline of pretrained encoders, a graph walk and a reranker, at self-reported positions that would be competitive on three Stanford leaderboards, with no generative model in the inference path. Equal link prediction did not imply equal usefulness for language: a RotatE embedding of the same width and near-equal link MRR took the same projection worse. What the representation did not learn to do, under two pre-registered attempts, is intersect constraints on its own; the exact walk still supplies that. The observation the evidence supports is modest and, we think, useful: a knowledge representation can be more than a link-prediction model, and which functions it supports is a property to measure, not to assume.

11 Reproducibility

The repository <https://github.com/jinxmcf/resonate> holds the shared model code at its root (`resonate.py`, the sparse shell `resonate_wiki.py`, `rowadagrad.py`) and one folder per board. `biokg/` contains both ladders’ trainers, the member and blend scripts, the receipts of the sparse-shell ladder (`results/sparse`) and of the first (`results/dense`), a `summarize.py` that recomputes every biokg statistic in this report from either, and a `verify.py` that asserts split disjointness and re-evaluates the released checkpoints of either ladder against their logs. `wikikg2/` contains the trainer, the member scripts, the learned combiner, the campaign scripts as run across three rented machines with the machine-specific paths removed, the receipts of every row and the cross-fitted validation logs, and `summarize_wiki.py`. A `VALIDATE.md` lists the commands a reviewer runs without retraining and what they printed on a fresh rented machine holding only the repository, the release and the OGB download: the statistics recompute from the receipts, all ten released biokg row-A models re-score to their logged test MRR, row B seed 0 rebuilt from its checkpoint reproduces the shipped receipt to the last digit, and the released wikikg2 teacher and student re-score to their logs. The releases carry the twenty sparse-shell `ogbl-biokg` checkpoints (row-A models re-saved in the dense format by an exact reinterpretation of the real-view table, and the $T = 2$ students), the first ladder’s twenty, and `ogbl-wikikg2`’s ten teachers and seven surviving students with bf16 tables.

Acknowledgements

The code, experiments and text were written with Claude Fable 5 (Anthropic) as pair programmer; the design, the protocol and every claim are the author’s responsibility.

References

- [1] OGB leaderboard for ogbl-wikikg2. https://ogb.stanford.edu/docs/leader_linkprop/, 2026. fetched 2026-09-04.
- [2] Derian Boer, Fabian Koch, and Stefan Kramer. Harnessing the power of semi-structured knowledge and LLMs with triplet-based prefiltering for question answering, 2024. arXiv:2409.00861.
- [3] Derian Boer, Stephen Roth, and Stefan Kramer. Focus, merge, rank: Improved question answering based on semi-structured knowledge bases, 2025. arXiv:2505.09246v1.
- [4] Derian Boer, Stephen Roth, and Stefan Kramer. Autofocus retrieval: An effective pipeline for multi-hop question answering with semi-structured knowledge. *Transactions on Machine Learning Research*, 2026. arXiv:2505.09246v4.
- [5] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Advances in Neural Information Processing Systems*, volume 26, 2013.
- [6] Linlin Chao, Jianshan He, Taifeng Wang, and Wei Chu. PairRE: Knowledge graph embeddings via paired relation vectors. In *Annual Meeting of the Association for Computational Linguistics*, 2021.
- [7] Yihong Chen, Pasquale Minervini, Sebastian Riedel, and Pontus Stenetorp. Relation prediction as an auxiliary training objective for improving multi-relational graph representations. In *Automated Knowledge Base Construction*, 2021.
- [8] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12, 2011.
- [9] Chang Gao, Chengjie Sun, Lili Shan, Lei Lin, and Mingjiang Wang. Rotate3D: Representing relations as rotations in three-dimensional space for knowledge graph embedding. In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management*, 2020.
- [10] Xiou Ge, Yun Cheng Wang, Bin Wang, and C.-C. Jay Kuo. Compound3d: Knowledge graph embedding with 3d compound geometric transformations. *arXiv preprint arXiv:2304.00378*, 2023.
- [11] Kelvin Guu, John Miller, and Percy Liang. Traversing knowledge graphs in vector space. In *Conference on Empirical Methods in Natural Language Processing*, 2015.
- [12] Daniel Scott Himmelstein, Antoine Lizée, Christine Hessler, Leo Brueggeman, Sabrina L. Chen, Dexter Hadley, Ari Green, Pouya Khankhanian, and Sergio E. Baranzini. Systematic integration of biomedical knowledge prioritizes drugs for repurposing. *eLife*, 6, 2017.
- [13] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

- [14] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. Open graph benchmark: Datasets for machine learning on graphs. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- [15] Vassilis N. Ioannidis, Xiang Song, Saurav Manchanda, Mufei Li, Xiaoqin Pan, Da Zheng, Xia Ning, Xiangxiang Zeng, and George Karypis. DRKG – drug repurposing knowledge graph for covid-19. <https://github.com/gnn4dr/DRKG/>, 2020.
- [16] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 2019.
- [17] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- [18] Timothée Lacroix, Nicolas Usunier, and Guillaume Obozinski. Canonical tensor decomposition for knowledge base completion. In *International Conference on Machine Learning*, 2018.
- [19] Adam Lerer, Ledell Wu, Jiajun Shen, Timothee Lacroix, Luca Wehrstedt, Abhijit Bose, and Alex Peysakhovich. PyTorch-BigGraph: A large-scale graph embedding system. In *Conference on Machine Learning and Systems*, 2019.
- [20] Hongzhu Li, Xiangrui Wang, Zhipeng Wang, et al. Stargraph: Knowledge graph embedding method via node association. *arXiv preprint arXiv:2205.14209*, 2022.
- [21] Millicent Li, Tongfei Chen, Benjamin Van Durme, and Patrick Xia. Multi-field adaptive retrieval. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.20056.
- [22] Haonan Lu, Hailin Hu, and Xiaodong Lin. DensE: An enhanced non-commutative representation for knowledge graph embedding with adaptive semantic hierarchy. *Neurocomputing*, 476, 2022.
- [23] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 2020.
- [24] OGB. OGB leaderboard for link property prediction: ogbl-biokg. https://ogb.stanford.edu/docs/leader_linkprop/#ogbl-biokg, 2026. Accessed 2026-09-02.
- [25] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [26] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *International Conference on Learning Representations*, 2019.
- [27] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. Complex embeddings for simple link prediction. In *International Conference on Machine Learning*, 2016.
- [28] Shirley Wu, Shiyu Zhao, Michihiro Yasunaga, Kexin Huang, Kaidi Cao, Qian Huang, Vassilis N. Ioannidis, Karthik Subbian, James Zou, and Jure Leskovec. Stark: Benchmarking llm retrieval on textual and relational knowledge bases. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

- [29] Yu Xia, Junda Wu, Sungchul Kim, Tong Yu, Ryan A. Rossi, Haoliang Wang, and Julian McAuley. Knowledge-aware query expansion with large language models for textual and relational retrieval. In *Proceedings of the 2025 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2025. arXiv:2410.13765.
- [30] Bishan Yang, Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. Embedding entities and relations for learning and inference in knowledge bases. In *International Conference on Learning Representations*, 2015.
- [31] Tong Yang, Yifei Wang, Long Sha, Jan Engelbrecht, and Pengyu Hong. Knowledgebra: An algebraic learning framework for knowledge graph. *Machine Learning and Knowledge Extraction*, 4(2), 2022.
- [32] Long Yu, Zhicong Luo, Huanyong Liu, Deng Lin, Hongzhu Li, and Yafeng Deng. TripleRE: Knowledge graph embeddings via tripled relation vectors. *arXiv preprint arXiv:2209.08271*, 2022.
- [33] Baole Yue, Xiaoming Yang, Chen Wu, et al. Interht: Knowledge graph embeddings by interaction between head and tail entities. *arXiv preprint arXiv:2202.04897*, 2022.
- [34] Ling Yue, Yongqi Zhang, Quanming Yao, Yiqun Li, Xian Wu, Ziheng Zhang, Zhenxi Lin, and Yefeng Zheng. Relation-aware ensemble learning for knowledge graph embedding. In *Conference on Empirical Methods in Natural Language Processing*, 2023.
- [35] Shuai Zhang, Yi Tay, Lina Yao, and Qi Liu. Quaternion knowledge graph embeddings. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [36] Xuanyu Zhang, Qing Yang, and Dongliang Xu. Trans: Transition-based knowledge graph embedding with synthetic relation representation. *arXiv preprint arXiv:2204.08401*, 2022.
- [37] Yongqi Zhang, Quanming Yao, Wenyuan Dai, and Lei Chen. AutoSF: Searching scoring functions for knowledge graph embedding. In *IEEE International Conference on Data Engineering*, 2020.
- [38] Yongqi Zhang, Quanming Yao, and James T. Kwok. Bilinear scoring function search for knowledge graph learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.
- [39] Yihua Zhu and Hidetoshi Shimodaira. Block-diagonal orthogonal relation and matrix entity for knowledge graph embedding. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024.